CHANDIGARH UNIVERSITY

APEX INSTITUTE OF TECHNOLOGY

EXPERIMENT 8

Subject: Advanced Database Management System

Submitted By:
Name: Akshra Ahuja
UID: 24BAI70523
Class: 24AIT_KRG_1B

Submitted To:
Mr. Alok Sir

Question 1

DEPARTMENT OF CAREER PLANNING & DEVELOPMENT

Problem Statement [Medium Level]

A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.
For every order, first calculate the **Gross Amount** using:
Gross Amount = Unit Price × Quantity
Then calculate the **Discount Amount** using:
Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the **Final Amount** using:

Final Amount = Gross Amount – Discount Amount

Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.
The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

1. Create View

```
(product_id, quantity, customer_id, discount_percent, order_date)
VALUES
('P1', 2, 'C1', 5, '2025-09-01'),
('P2', 1, 'C2', 10, '2025-09-02'),
('P3', 3, 'C3', 0, '2025-09-03'),
('P4', 1, 'C4', 8, '2025-09-04'),
('P5', 1, 'C5', 12, '2025-09-05'),
('P1', 1, 'C2', 5, '2025-09-06'),
('P3', 2, 'C1', 0, '2025-09-07');

create or replace view cv_employee
as
Select s.order_id, s.order_date, c.full_name, p.product_name, round((p.unit_price*s.quantity)-(p.unit_price*s.quantity*s.discount_per
from sales_orders s
join product_catalog p
on p.product_id=s.product_id
join customer_master c
on c.customer_id=s.customer_id;


Select * from cv_employee;
```

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	total_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

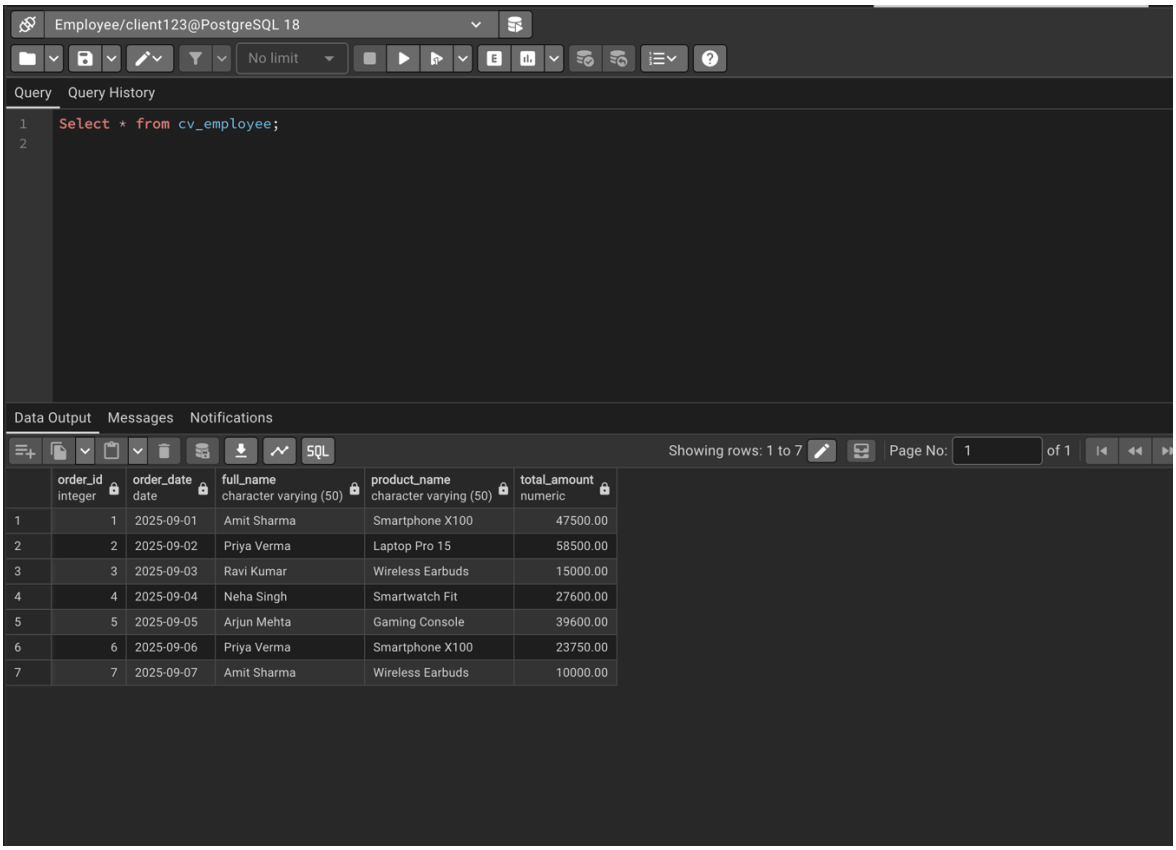
2. Create user and grant select permission

```
create user client123
password '12345';

grant select on cv_employee to client123;
```

Data Output | Messages | Notifications

```
GRANT

Query returned successfully in 62 msec.
```

3. Selecting from view



	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	total_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Question 2

DEPARTMENT OF CAREER PLANNING & DEVELOPMENT

NAAC GRADE A+

RANKED #20 AMONGST TOP UNIVERSITIES IN INDIA

Problem Statement [Hard Level]

An e-commerce company wants to regularly analyze the performance of its **product categories**.
The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple **JOINs, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements**.
Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.
Create a PostgreSQL **Materialized View** named: mv_category_sales

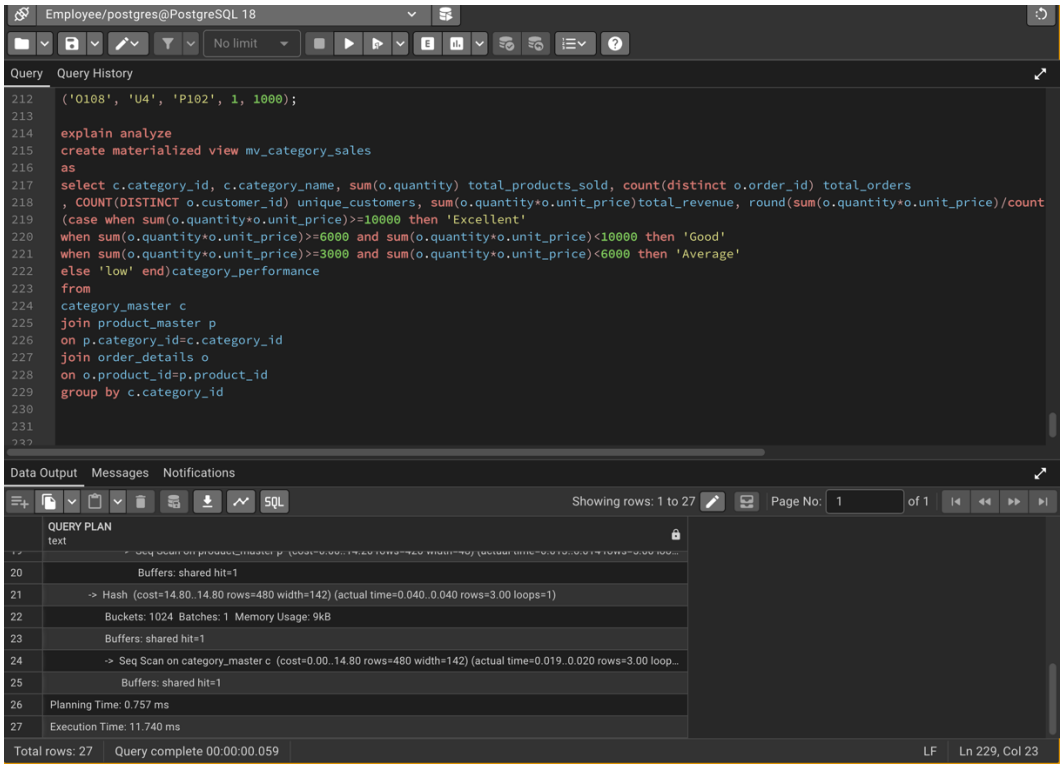
The report must contain:

- Category ID
- Category Name
- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

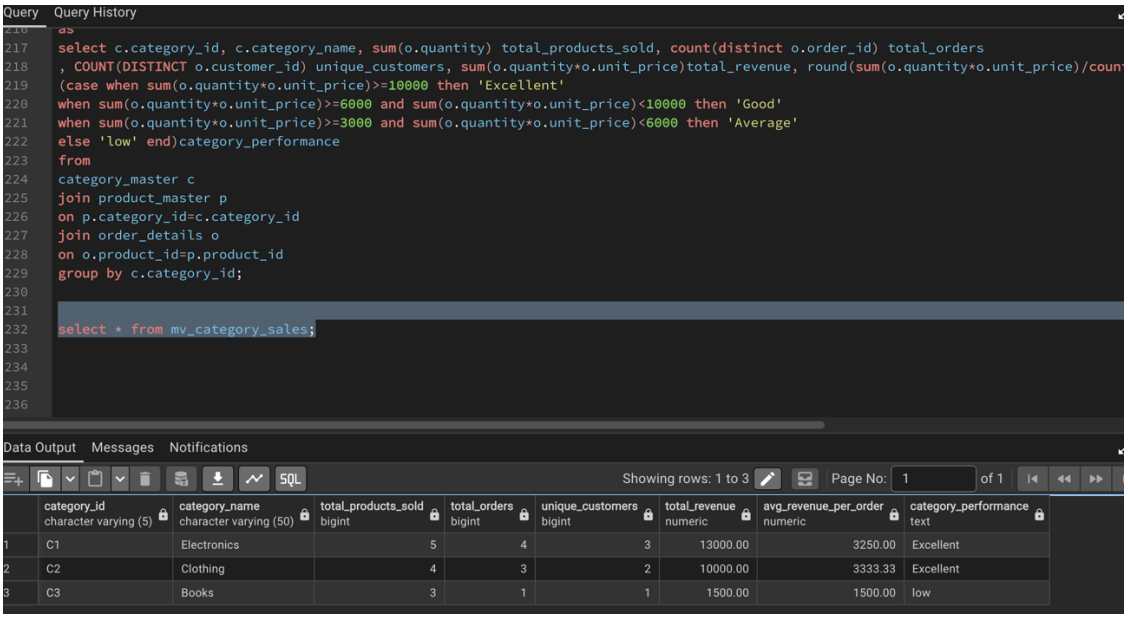
Also demonstrate that when new order data is inserted, the Materialized View does **not automatically reflect the change** and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

1. Check time for execution while creating the materialized view



2. Execute the materialized view



3. Analyse time for execution for data retrieval